Inheritance

What is Inheritance?

- A fundamental OOP concept in Java.
- Mechanism where one class (`subclass` or `child class`) inherits features (fields and methods) from another class (`superclass` or `parent class`).
- Promotes **code reusability**, enables **method overriding**, and supports **polymorphism**.
- Implemented using the `extends` keyword.

Why Use Inheritance?

- **Code Reusability:** Child classes can directly use parent class code, reducing duplication.
- **Method Overriding:** Allows subclasses to provide specific implementations for methods defined in the superclass, enabling runtime polymorphism.
- **Abstraction:** Supports abstraction by defining common interfaces and behaviors in parent classes.
- **Structured Organization:** Organizes classes in a hierarchical manner, improving readability and maintainability.

How Inheritance Works (The "is-a" Relationship)

- The `extends` keyword establishes an **"is-a" relationship** (e.g., `MountainBike` *is a* `Bicycle`).
- Subclasses inherit all non-private members (fields and methods) of the superclass.
- Subclasses can override inherited methods or add new fields and methods.

Syntax:

```
class ChildClass extends ParentClass {
    // Additional fields and methods
}
```

Example: `Bicycle` and `MountainBike`

```
// base class
class Bicycle {
    public int gear;
    public int speed;

    public Bicycle(int gear, int speed) {
        this.gear = gear;
        this.speed = speed;
    }

    public void applyBrake(int decrement) { speed -= decrement; }
    public void speedUp(int increment) { speed += increment; }
```

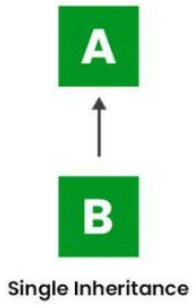
```
    public String toString() {
        return ("No of gears are " + gear + "\n" + "speed of bicycle is " +
speed);
    }
}

// derived class
class MountainBike extends Bicycle {
    public int seatHeight;

    public MountainBike(int gear, int speed, int startHeight) {
        super(gear, speed); // Invoking base-class constructor
        seatHeight = startHeight;
    }

    public void setHeight(int newValue) { seatHeight = newValue; }

    @Override
    public String toString() {
        return (super.toString() + "\nseat height is " + seatHeight);
    }
}

// driver class
public class Test {
    public static void main(String args[]) {
        MountainBike mb = new MountainBike(3, 100, 25);
        System.out.println(mb.toString());
    }
}
```

Output:

```
No of gears are 3
speed of bicycle is 100
seat height is 25
```

Types of Inheritance in Java

Single Inheritance: A subclass inherits from only one superclass.



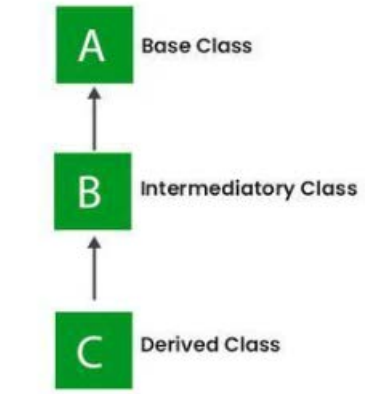
Single Inheritance

```
class One {
   public void print_geek() { System.out.println("Geeks"); }
}
class Two extends One {
    public void print_for() { System.out.println("for"); }
}
public class Main {
    public static void main(String[] args) {
        Two g = new Two();
        g.print_geek();
        g.print_for();
        g.print_geek();
    }
}
```

Output:

```
Geeks
for
Geeks
```

Multilevel Inheritance: A derived class inherits from a base class, which itself is a derived class.



Multilevel Inheritance

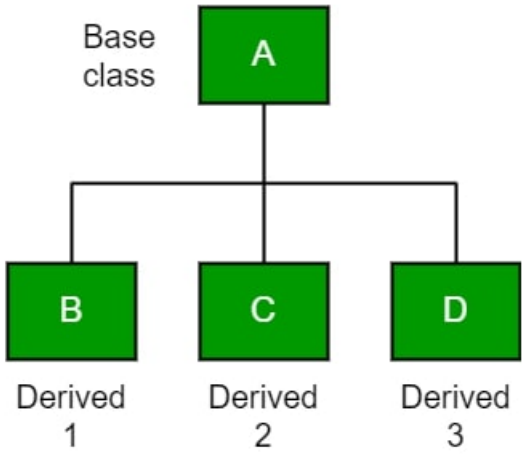
```
class One { public void print_geek() { System.out.println("Geeks"); } }
class Two extends One { public void print_for() {
System.out.println("for"); } }
class Three extends Two { public void print_lastgeek() {
System.out.println("Geeks"); } }
public class Main {
    public static void main(String[] args) {
        Three g = new Three();
        g.print_geek();
        g.print_for();
        g.print_lastgeek();
    }
```

```
}
```

Output:

```
Geeks
for
Geeks
```

Hierarchical Inheritance: One superclass is inherited by multiple subclasses.

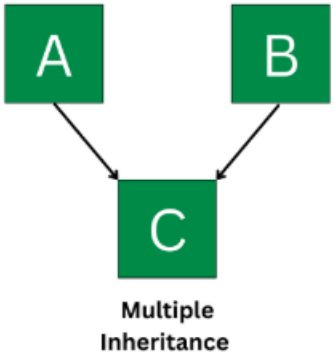



```
class A { public void print_A() { System.out.println("Class A"); } }
class B extends A { public void print_B() { System.out.println("Class
B"); } }
class C extends A { public void print_C() { System.out.println("Class
C"); } }
class D extends A { public void print_D() { System.out.println("Class
D"); } }
public class Test {
    public static void main(String[] args) {
        B obj_B = new B(); obj_B.print_A(); obj_B.print_B();
        C obj_C = new C(); obj_C.print_A(); obj_C.print_C();
        D obj_D = new D(); obj_D.print_A(); obj_D.print_D();
    }
}
```

Output:

```
Class A
Class B
Class A
Class C
Class A
Class D
```

Multiple Inheritance (via Interfaces): One class implements multiple interfaces (Java classes do not support multiple inheritance directly).

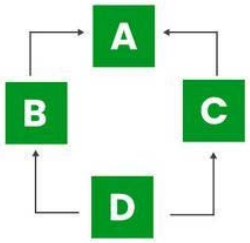



```
interface Coder { void writeCode(); }
interface Tester { void testCode(); }
class DevOpsEngineer implements Coder, Tester {
    @Override public void writeCode() { System.out.println("DevOps
Engineer writes automation scripts."); }
    @Override public void testCode() { System.out.println("DevOps
Engineer tests deployment pipelines."); }
    public void deploy() { System.out.println("DevOps Engineer deploys
code to cloud."); }
}
public class Main {
    public static void main(String[] args) {
        DevOpsEngineer devOps = new DevOpsEngineer();
        devOps.writeCode();
        devOps.testCode();
        devOps.deploy();
    }
}
```

Output:

```
DevOps Engineer writes automation scripts.
DevOps Engineer tests deployment pipelines.
DevOps Engineer deploys code to cloud.
```

Hybrid Inheritance: A mix of two or more of the above types. Can involve classes and/or interfaces.



Hybrid Inheritance

Subclass Capabilities

In a subclass, you have several powerful options for managing inherited members and introducing new ones:

- **Utilize Inherited Members -** You can directly use fields and methods inherited from the superclass as they are.
- **Add New Members -** You're able to declare completely **new fields and methods** that are specific to the subclass and not present in the superclass.
- **Override Instance Methods -** For instance methods, you can **override** them by creating a new method in the subclass with the exact same signature as one in the superclass. This allows you to change its behavior in the subclass (e.g., `toString()` method).
- **Hide Static Methods -** Similarly, you can **hide** static methods by declaring a new static method in the subclass with the same signature.
- **Constructors -** Subclass constructors can **invoke the superclass constructor**, either automatically or explicitly using the `super` keyword, to ensure proper initialization.

Advantages of Inheritance

- **Code Reusability:** Reduces redundant code.
- **Abstraction:** Supports abstract classes defining common interfaces.
- **Class Hierarchy:** Models real-world objects and relationships.
- **Polymorphism:** Allows objects to take on multiple forms through method overriding.

Disadvantages of Inheritance

- **Complexity:** Can make code harder to understand, especially with deep hierarchies.
- **Tight Coupling:** Creates strong dependencies between superclass and subclass, making changes potentially impactful.

Key Takeaways

- Every Java class (except `Object`) implicitly extends `Object`.
- A class can only have **one direct superclass** (single inheritance for classes).

- Constructors are **not inherited** by subclasses, but superclass constructors can be invoked using `super()`.
- **Private members are not inherited** by subclasses, but can be accessed via public/protected getters/setters from the superclass.